Informe técnico — TP API REST de Personas y Pizzas

Alumno: Matías Messina · **Materia:** Taller de Programación II · **Universidad FASTA Fecha:** 9/11/2026

1. Parte 1 — Puesta en marcha y diagnóstico

Se levantó el proyecto siguiendo el Manual, se verificó que respondieran `http://localhost:8000/` y `http://localhost:8000/api/ui/`, y se probaron los 9 endpoints desde Swagger UI.

1.1 Contrato declarado contra comportamiento real

#	Método	Ruta	Declara `swagger.yml`	Devuelve la API	¿Coincide?	Captura
1	GET	`/api/people`	200	200	Sí	`01-get-people.png`
2	POST	`/api/people`	201	201	Sí	`03-post-people-messina-201.png`
3	GET	`/api/people/{lname}`	200	200	Sí	`02-get-people-gomez.png`
4	PUT	`/api/people/{lname}`	200	**201**	**No**	`07-put-people-201.png`
5	DELETE	`/api/people/{lname}`	204	**200 + texto plano**	**No**	`09-delete-people-200.png`
6	POST	`/api/pizzas`	201	201	Sí	`04-post-pizzas-201.png`
7	GET	`/api/pizzas/{pizza_id}`	200	200	Sí	`05-get-pizza-200.png`
8	PUT	`/api/pizzas/{pizza_id}`	200	**201**	**No**	`06-put-pizza-201.png`
9	DELETE	`/api/pizzas/{pizza_id}`	204	204 (con cuerpo)	**Parcial**	`08-delete-pizza-204.png`

1.2 Diferencias detectadas

Diferencia 1 — Los `PUT` devuelven 201 en vez de 200. Afecta a `people.update` y `pizzas.update`. El contrato declara `200` en ambos casos, pero el código termina con `return ..., 201`. *Evidencia:* `PUT /api/people/Diaz` con `{"lname": "Diaz", "fname": "Lucia Maria"}` devolvió `201 CREATED` (captura 07); `PUT /api/pizzas/{id}` con `{"content": "Pizza Napolitana"}` devolvió `201 CREATED` (captura 06).

Diferencia 2 — `DELETE /api/people/{lname}` devuelve 200 con texto. El contrato declara `204`, pero el código hace `make_response(f"{lname} successfully deleted", 200)`. *Evidencia:* `DELETE /api/people/Diaz` devolvió `200 OK` con el cuerpo `Diaz successfully deleted` (captura 09).

Diferencia 3 — `DELETE /api/pizzas/{pizza_id}` arma un cuerpo para un 204. El código es `make_response(f"{pizza_id} successfully deleted", 204)`. El código de estado coincide con el contrato, pero la especificación HTTP establece que un `204 No Content` **no puede tener cuerpo**: Werkzeug lo descarta, así que ese texto nunca llega al cliente. Es una inconsistencia del código consigo mismo.

Hallazgo adicional — El contrato no declara ningún código de error. Revisando las 9 operaciones de `swagger.yml`, las únicas respuestas declaradas son `200`, `201` y `204`. Sin embargo la API devuelve en la práctica `404` (recurso inexistente), `406` (apellido duplicado) y `400` (campo obligatorio faltante, rechazado por Connexion antes de llegar a la función). Esta diferencia no figura en la lista de observaciones del Manual. *Evidencia:* capturas `10-get-people-inexistente-404.png`, `11-post-people-duplicado-406.png` y `12-post-people-sin-lname-400.png`.

---

2. Parte 2 — Alinear contrato y código

2.1 Decisiones y justificación

En las tres diferencias se decidió **corregir el código y no el contrato**, porque en los tres casos el contrato describe el comportamiento correcto según la semántica de HTTP.

#	Caso	Decisión	Justificación
1	`PUT` devolvía 201	Código → `200`	`201 Created` indica que se creó un recurso nuevo. Un `PUT` que modifica un recurso existente no crea nada: corresponde `200 OK`.
2	`DELETE /people` devolvía 200 + texto	Código → `204`	El borrado no tiene contenido útil que devolver. Además `DELETE /pizzas` ya devolvía `204`: unificarlos deja la API consistente consigo misma.
3	`DELETE /pizzas` mandaba cuerpo con un 204	Código → `204` vacío	Un `204 No Content` no puede llevar cuerpo. El texto nunca llegaba al cliente.

Criterio general aplicado: cuando el contrato es correcto según el estándar HTTP, se corrige el código; el contrato solo se modifica cuando describe mal algo que está bien implementado. En este proyecto no se presentó el segundo caso.

Evidencia de las correcciones: capturas `13-put-people-200-corregido.png`, `14-put-pizza-200-corregido.png` y `15-delete-people-204-corregido.png`.

2.2 Esquema `Pizza` en `components/schemas`

Los endpoints de pizzas definían sus esquemas **en línea**, duplicando la descripción de los campos. Se movieron a `components/schemas` y se referencian con `$ref`, de modo que **no queda ningún esquema inline** en el contrato.

Se definieron **dos** esquemas en lugar de uno, por una diferencia real entre las operaciones:

- **`Pizza`** (usado en `POST /api/pizzas`) exige `person_id`, porque sin él no se sabe de quién es la pizza.
- **`PizzaUpdate`** (usado en `PUT /api/pizzas/{pizza_id}`) no lo exige, porque la pizza ya está asociada a una persona y la operación no cambia esa relación.

Reutilizar un único esquema en ambas operaciones habría obligado al `PUT` a recibir un `person_id` que ignora, cambiando el comportamiento de la API sin motivo.

Efecto lateral positivo: antes, un `PUT /api/pizzas/1` con el cuerpo `{}` pasaba la validación del contrato y fallaba con un **500**, porque `content` es `nullable=False` en la base. Al declarar `content` como obligatorio en `PizzaUpdate`, Connexion lo rechaza antes y devuelve un **400**.

2.3 Códigos de error en el contrato

Se agregaron a las 12 operaciones los códigos que la API devuelve realmente (`400`, `404` y `406`), cada uno con su descripción en español. Swagger UI ahora documenta tanto el camino feliz como los de error.

2.4 Actualización de APIs heredadas

Qué	Dónde	Por qué
`Query.get()` → `db.session.get(Modelo, id)`	`pizzas.py`, 4 llamadas	`Query.get()` es API heredada en SQLAlchemy 2.x
`datetime.utcnow` → `datetime.now(timezone.utc)`	`models.py`, ambas clases	`utcnow` está deprecado desde Python 3.12: devuelve una fecha sin zona horaria que pretende ser UTC

El reemplazo se encapsuló en una función `ahora()` porque SQLAlchemy necesita **un callable** que se evalúe en cada inserción. Pasando el valor ya calculado, todas las filas quedarían con la hora de arranque del programa.

---

3. Parte 3 — Endpoints nuevos

Método	Ruta	Comportamiento	Errores
`GET`	`/api/pizzas`	Lista todas las pizzas, de la más nueva a la más vieja. Con `?person_id=` devuelve solo las de esa persona	`404` si el `person_id` no existe
`GET`	`/api/people/{lname}/pizzas`	Lista solo las pizzas de esa persona	`404` si la persona no existe

Método	Ruta	Comportamiento	Errores
`GET`	`/api/people/{lname}/resumen`	Devuelve `lname`, `fname`, `cantidad_pizzas` y `total_gastado`	`404` si la persona no existe

Los tres se declararon primero en `swagger.yml` y después se implementó la función, siguiendo el enfoque *contract first*.

Detalle de implementación. En `GET /api/pizzas`, el parámetro `person_id` llega como argumento de la función con valor por defecto `None`, porque Connexion mapea los *query params* del contrato a argumentos de Python y no los pasa cuando no vienen en la URL.

`read_resumen` es la única función que **no usa un esquema de Marshmallow**: devuelve un diccionario armado a mano, porque el resumen no es una entidad de la base sino un cálculo derivado. Flask lo serializa a JSON automáticamente.

Evidencia: capturas.

Verificación	Resultado
`GET /api/pizzas`	7 pizzas
`GET /api/pizzas?person_id=2`	2 pizzas (las de Gomez)
`GET /api/people/Sanchez/resumen`	`cantidad_pizzas: 3`, `total_gastado: 24800.0`
`GET /api/people/Gomez/resumen`	`cantidad_pizzas: 2`, `total_gastado: 17300.0`
`GET /api/people/Lopez/resumen`	`cantidad_pizzas: 2`, `total_gastado: 15300.0`

---

4. Parte 4 — Modelo extendido

4.1 Campos nuevos

Campo	Tipo en el modelo	Validación en el contrato
`price`	`db.Float`, `nullable=False`	`type: number`, `minimum: 0`, `exclusiveMinimum: true`
`size`	`db.String(10)`, `nullable=False`	`type: string`, `enum: [chica, mediana, grande]`

El par `minimum: 0` + `exclusiveMinimum: true` es la forma de expresar "*estrictamente mayor que 0*" en **OpenAPI 3.0**: solo `minimum: 0` aceptaría una pizza de precio cero. En OpenAPI 3.1 la sintaxis cambia a `exclusiveMinimum: 0`, pero este contrato declara `openapi: 3.0.0`.

4.2 Cambios en el código

`create()` **no requirió modificaciones**: `PizzaSchema` hereda de `SQLAlchemyAutoSchema`, que genera sus campos automáticamente a partir de las columnas del modelo, de modo que reconoció `price` y `size` sin declararlos. `update()` sí, porque allí la asignación está escrita campo por campo.

`PizzaUpdate` exige los tres campos (`content`, `price`, `size`), por lo que un `PUT` con un solo campo ahora devuelve `400`. **Es un cambio deliberado:** un `PUT` reemplaza la representación completa del recurso; para modificar un solo campo el verbo correcto sería `PATCH`. Esto modifica un comportamiento que el Manual documentaba.

4.3 Datos de ejemplo

Todas las pizzas de `build_database.py` tienen ahora precio y tamaño, y se corrigieron los errores de tipeo: **"Piza Cuatro Quesos"** → **"Pizza Cuatro Quesos"**, **"Pizza roqefort"** → **"Pizza Roquefort"**, **"Pizza Peperoni"** → **"Pizza Pepperoni"**, y los acentos de **"Jamón Crudo y Rúcula"**.

4.4 Verificación de los 400

Cuerpo enviado a `POST /api/pizzas`	**Resultado**	**Captura**
`price: 8200, size: "grande"`	`201`	`20-post-pizza-price-size-201.png`
`price: -100`	`400`	
`price: 0`	`400`	—
`size: "familiar"`	`400`	
sin `price`	`400`	

Estos `400` los genera **Connexion a partir del contrato**, no el código de la aplicación: el mensaje viene con el formato de la librería. El esquema `Pizza` con su `enum` y su `minimum` se puede ver en la captura `21-swagger-schema.png`.

---

5. Parte 5 — Calidad: errores y pruebas

5.1 Unificación del idioma

Los ocho mensajes de error de `people.py` y `pizzas.py` pasaron al español:

Antes	**Ahora**
`Person with last name {lname} already exists`	`Ya existe una persona con el apellido {lname}`
`Person with last name {lname} not found`	`No existe una persona con el apellido {lname}`
`Pizza with ID {pizza_id} not found`	`No existe una pizza con el ID {pizza_id}`
`Person not found for ID: {person_id}`	`No existe una persona con el ID {person_id}`

5.2 Validación de campos vacíos

La validación quedó en **dos niveles**, por una razón concreta:

1. **En el contrato** (`minLength: 1` en `lname` y `content`) se rechaza la cadena de longitud 0.
2. **En el código** (`.strip()` al inicio de `create` y `update`) se rechaza además la cadena que solo tiene espacios: para OpenAPI, `"  "` es un string válido de longitud 3 y pasa el `minLength`.

El código además **normaliza** el valor antes de guardarlo (`person["lname"] = lname`). Sin eso, `"Gomez "` con un espacio final se guardaría como una persona distinta de `"Gomez"` y una consulta posterior por apellido no la encontraría — que es exactamente el síntoma que el Manual lista como *"404 al consultar una persona que sí existe"*.

5.3 Pruebas automáticas

Se escribieron **24 pruebas** en `tests/test_api.py` usando el cliente de pruebas de Flask, sobre el mínimo de 10 que pedía la consigna.

Grupo	Cubre
Personas (7)	Listar, crear, duplicado `406`, inexistente `404`, apellido vacío `400`, `PUT` → `200`, `DELETE` → `204`
Cascada (1)	Al borrar una persona desaparecen sus pizzas
Pizzas (8)	Crear, persona inexistente `404`, precio negativo `400`, precio cero `400`, tamaño inválido `400`, descripción vacía `400`, `PUT` → `200`, inexistente `404`
Endpoints nuevos (8)	Listado completo, filtro por `person_id`, filtro con id inexistente `404`, pizzas de una persona, persona inexistente `404`, resumen, resumen de una persona sin pizzas, resumen de persona inexistente `404`

Aislamiento. Para que las pruebas no destruyan `people.db`, se modificó `config.py` para que lea la ruta de la base desde la variable de entorno `DATABASE_URI`, con `people.db` como valor por defecto. `conftest.py` la apunta a una base temporal **antes** de importar la aplicación, porque Flask-SQLAlchemy crea el motor en el momento del import.

Diseño de los datos de prueba. La fixture carga tres personas: una con dos pizzas, una con una, y **una sin ninguna**. Esta última existe para verificar que el resumen de alguien que no pidió nada devuelve `0` y no falla.

Pruebas de regresión. `test_actualizar_persona_da_200`, `test_borrar_persona_da_204` y `test_actualizar_pizza_da_200` fallarían si alguien revirtiera las correcciones de la Parte 2. Las pruebas no solo verifican que la API funciona: fijan las decisiones tomadas.

Resultado: `24 passed` — captura `22-pytest-24-passed.png`.

5.4 Advertencias de deprecación

`pytest` reporta advertencias de `jsonschema` (`draft4_format_checker`, `RefResolver`) y de `asyncio.iscoroutinefunction`. **No provienen del código del proyecto** sino de Connexion 2 corriendo sobre Python 3.14, y no se pueden eliminar sin migrar a Connexion 3, que las pautas del trabajo no permiten.

Una tercera advertencia, `x-body-name within the requestBody schema will be deprecated`, confirma que Connexion está leyendo correctamente el `x-body-name` ubicado junto al `$ref` — que era el punto dudoso al reutilizar el esquema `Pizza`.

6. Frontend

Se desarrolló un frontend en **React con Next.js** (App Router) que consume la API, como complemento a lo pedido en el enunciado.

Ruta	Qué muestra	Endpoints que usa
`/`	Lista de clientes, alta y borrado de personas	`GET/POST/DELETE /api/people`
`/personas/{lname}`	Detalle, resumen de consumo, alta y borrado de pizzas	`GET /api/people/{lname}`, `GET .../resumen`, `POST/DELETE /api/pizzas`
`/pizzas`	Todas las pizzas con filtro por persona	`GET /api/pizzas`, `GET /api/pizzas?person_id=`

Componentes identificados. El criterio para separar un componente fue que se repitiera, que se usara en más de una pantalla, o que tuviera estado propio:

Componente	Por qué
`TarjetaPersona`	Se repite una vez por persona en la lista
`TarjetaPizza`	**Se usa en dos pantallas distintas** (detalle y listado general)
`FormularioPersona`, `FormularioPizza`	Tienen estado propio (los campos del formulario)
`Resumen`	Unidad conceptual, aunque aparezca una sola vez
`Mensaje`, `Cargando`, `BotonBorrar`	Se usan en las tres pantallas

`TarjetaPizza` es el caso que mejor justifica la decisión: recibe `onBorrar` solo en la pantalla de detalle y `duenio` solo en el listado general, y se adapta a los dos contextos con renderizado condicional, sin duplicar el marcado.

CORS. El navegador bloquea las peticiones de `localhost:3000` a `localhost:8000` por ser orígenes distintos. Se habilitó `flask-cors` en `config.py` **restringido al origen del frontend de desarrollo y solo bajo `/api/`**, en lugar de abrir la API a cualquier origen.

Gracias a que el cliente de la API lee el campo `detail` de la respuesta de error, **los mensajes en español escritos en la Parte 5 se muestran directamente en la interfaz**.

---

7. Dificultades encontradas

1. El enunciado tiene una dependencia cruzada entre las Partes 3 y 4. `GET /people/{lname}/resumen` (Parte 3) debe devolver `total_gastado`, la suma de los precios, pero el campo `price` recién se agrega en la Parte 4. Fue necesario adelantar las dos columnas del modelo para poder implementar el endpoint, y dejar para la Parte 4 la validación en el contrato y su verificación.

2. Connexion lee `swagger.yml` una sola vez, al arrancar. El recargador automático de Flask vigila los archivos `.py`, no el YAML, y Swagger UI además cachea el `openapi.json` en el navegador. Al agregar el esquema `Pizza` la interfaz seguía mostrando solo `Person`, y el problema no era el contrato sino que el

servidor servía la versión anterior. Hubo que detener el servidor por completo y forzar la recarga del navegador.

3. Un `datetime.utcnow` olvidado se detectó por las advertencias. Tras cambiar `models.py`, `pytest` seguía reportando 75 advertencias de `utcnow is deprecated`. La advertencia solo se dispara cuando el *default* de la columna se ejecuta, lo que indicaba que una de las dos clases conservaba el original. Leer las advertencias, y no solo los fallos, sirvió para encontrarlo.

4. En el frontend, `components/` y `lib/` no pueden ir dentro de `app/`. En el App Router cada carpeta dentro de `app/` se interpreta como una ruta. Haber colocado `api.js` en `src/app/lib/` rompía el alias `@/lib/api`, que resuelve a `src/lib/`.

(Agregar acá cualquier otra dificultad propia: errores de tipeo que costaron encontrar, problemas de entorno, etc.)

---

8. Declaración de herramientas de apoyo

Para la realización de este trabajo se utilizó un asistente de inteligencia artificial (Claude) como herramienta de apoyo, principalmente para: revisar el contrato `swagger.yml` contra el código en busca de inconsistencias, discutir las decisiones de diseño documentadas en la sección 2, y redactar la estructura inicial de las pruebas y del frontend. Las decisiones de implementación, la verificación de cada endpoint y la integración final fueron realizadas y comprobadas por mi.

9. Historial de Git

Commit	Parte
`commit inicial del proyecto`	Punto de partida
`docs: diagnostico de la parte 1 con capturas`	Parte 1
`fix: alinea contrato y codigo, esquema Pizza y API moderna de SQLAlchemy`	Parte 2
`feat: listado de pizzas, pizzas por persona y resumen de consumo`	Parte 3
`feat: valida price y size desde el contrato y los aplica en update`	Parte 4
`test: 24 pruebas con pytest, mensajes en espanol y validacion de vacios`	Parte 5
`feat: frontend en Next.js que consume la API`	Frontend
`docs: README e informe`	Documentación